



Reading your run

When a run ends we post the result to your callback URL, and that message **leads with a web address**. Open it.

The address

It is an address, not a login. No password, no account: whoever holds the link can read your runs. Treat it as you would a password.

We keep the **last 5 runs of each stage**, 15 in all. Older ones drop off.

How it is arranged

A run is not a flat list of calls. It nests:

```
run
├─ problem          what was asked, and what it was worth
│   └─ attempt      up to three; the best one counts
│       └─ call       in order, with everything we recorded
```

Open the problem that lost points, then its attempts, then the calls. That is the

whole method.

Finding out why something scored what it did

The problem shows what it was worth and what it got.

The attempts — up to three, best counts. Two things worth knowing:

- We stop early once an attempt takes full marks, so a problem that went right often shows one attempt, not three. Nothing is missing.
- Seeing the same question three times means the first two did not score. That is expected, not a bug.

The calls. Every call we forwarded to you, in order, each with:

the reason	why our android said it was making this call, in its own words
what we sent	the arguments your server received
what you answered	your response as we got it
what we did with it	accepted, refused, discarded — and why
how long you took	your own time, not counting ours
what it cost	the tokens your response occupied

The reason is the most under-used thing on the page. It is the android saying what it is about to do and why. When something goes wrong that your own logs cannot explain, that is usually where you see it; reaching for a tool you never exposed, or asking for the wrong thing in exactly the right shape.

What every call cost

The tokens your response took up are shown on **every** call, not just the ones that broke a limit. A limit you only hear about after crossing it is no use; the point is seeing how much room you had while things were still fine.

Where a response *was* discarded for being too long, you also get the beginning of what you sent, so you can see *what* was too long rather than only *that* it was.

Problems you were never asked

Some problems show as not asked rather than failed. Related problems are asked in order, and one that scores nothing ends its line — the rest are never put to your server.

The page names the problem whose zero closed the line, and it always names one that was really asked and really scored zero. So it points at a cause you can go and read.

A problem that was never asked shows no prompt.

Runs that never connected

If we could not reach you at all, there is still a page, and it lists **every address we tried and how each failed**. There is no transcript, because the run ended before the first question.

[/qna](#) is the page for that one.

What your server looked like to us

The page shows the tools you advertised, **as we read them**; names, descriptions, schemas.

Check it once, early. It is the only place you see your server the way the android sees it, and a description that reads well to you and badly to a model costs you in a way nothing else will show.

If a tool broke one of the limits on the tools you expose, the page names which tool and which limit. Those limits are at [/limits](#).

What you will never find there

- **The expected answer.**
 - **Anyone else's run.**
-

"Truncated"

Very long responses get capped and the record says so. That is a normal state, not an error; the details of each call survive even when big response bodies do not.

If it happens on a run you care about, it usually means your responses are much larger than they need to be, which is worth knowing anyway.